

learn-dev

Plateforme web d'apprentissage de la programmation

Eric Bouchut – Titre professionnel Développeur Web et Web Mobile (DWWM)

Soutenance – RNCP37674 – session août 2026

Introduction

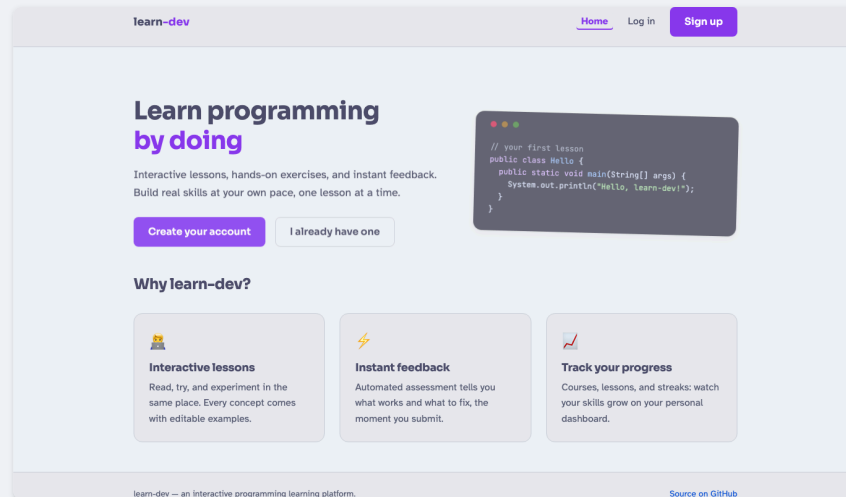
Le besoin

- Apprendre à programmer suppose un **parcours structuré** : du contenu organisé, une progression lisible, un espace où un formateur publie et fait évoluer ses cours.
- **Problème** : offrir un environnement multi-rôles simple, **sûr** et **accessible**.
- **Solution, en une phrase** :

learn-dev est une plateforme web où des formateurs publient des cours composés de leçons en Markdown, et où des étudiants s'inscrivent et les suivent.

learn-dev en bref

- **Trois rôles** : étudiant (**STUDENT**), formateur (**INSTRUCTOR**), administrateur (**ADMIN**).
- **Étudiant** : catalogue, inscription à un cours, lecture des leçons.
- **Formateur** : création de cours/leçons, cycle de vie de publication, gestion des inscrits.
- **Administrateur** : gestion des comptes, modération des contenus.



Conception

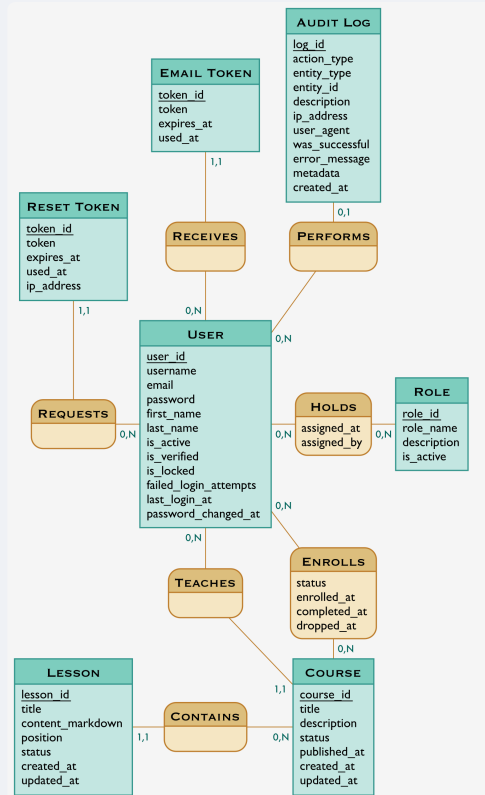
Gestion de projet

- **Git par fonctionnalité** : une branche par feature, convention *Conventional Commits*.
- **Développement incrémental** : auth → domaine cours → authoring → administration.
- **14 décisions d'architecture (ADR)** tracées (format MADR).
- Principe **YAGNI** : on ne construit que le nécessaire à la v1.

Cahier des charges & personas

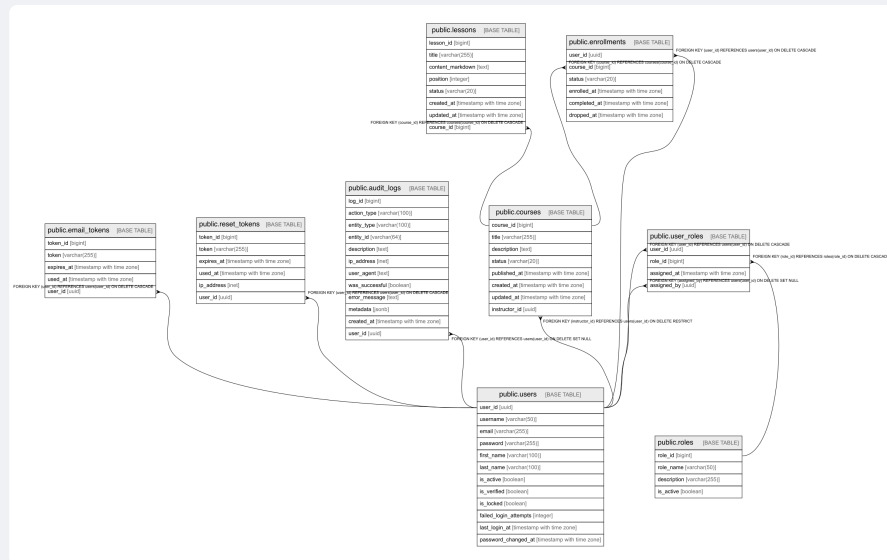
- **Objectifs** : s'inscrire et suivre des cours, publier/gérer des cours, administrer.
- **Personas** :
 - **Léa** (étudiante) – suivre des cours, sur mobile comme sur ordinateur.
 - **Marc** (formateur) – rédiger en Markdown, gérer publication et inscrits.
 - **Awa** (admin) – gérer les comptes et modérer en toute sécurité.
- **Non fonctionnel** : sécurité, accessibilité RGAA, responsive, tests, CI.

Base de données – modèle conceptuel (CP5)



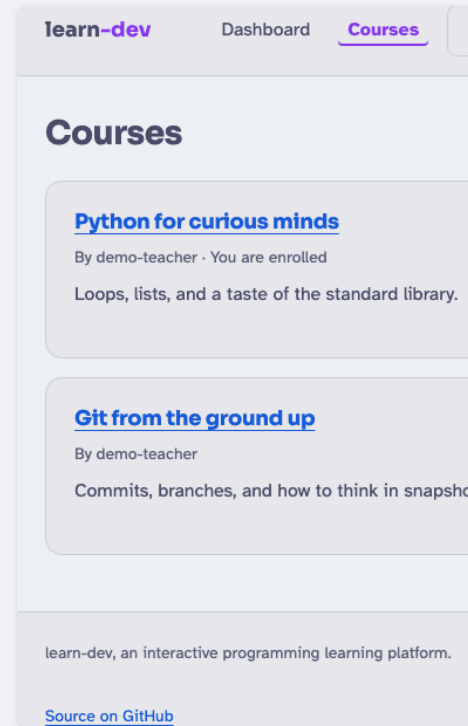
Base de données – schéma physique & choix (CP5)

- PostgreSQL 17, 9 tables, migrations **Liquibase** appliquées au démarrage.
- Clé **UUID** pour les utilisateurs (anti-IDOR) ; **BIGINT** ailleurs ; clés composites.
- Contraintes : **UNIQUE** , **CHECK** , **FK ON DELETE CASCADE** ; Hibernate en **validate** .



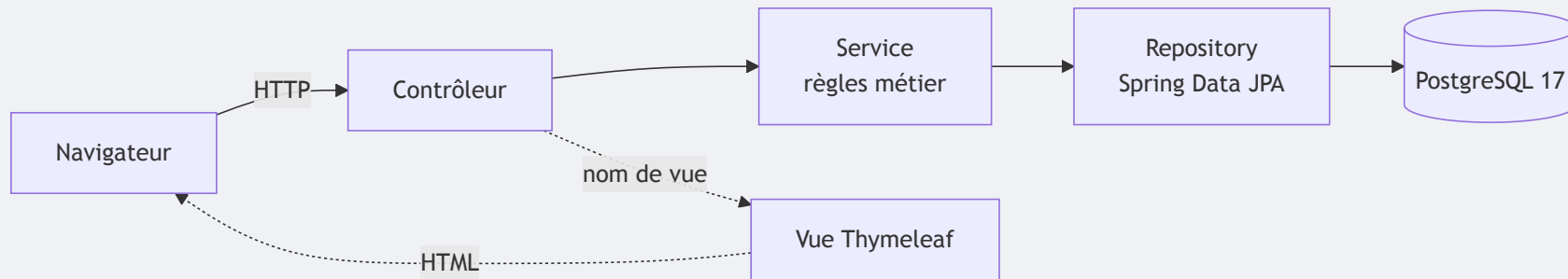
Maquettage & accessibilité (CP2)

- **Maquettes Figma** + prototypes HTML ; charte, deux thèmes (Catppuccin / Soft Paper).
- **Responsive** : versions web (bureau) **et web mobile** (points de rupture en `rem`).
- **Accessibilité RGAA / WCAG AA** : structure sémantique, *skip link*, contrastes calculés (min. 4,73:1), police **Atkinson Hyperlegible**.



Environnement technique (CP1)

- **Java 21 + Spring Boot 3.5** (Web MVC, Security, Data JPA, Validation, Mail).
- **Thymeleaf** (vues), **PostgreSQL 17 + Liquibase, commonmark-java + jsoup**.
- **Podman / Docker Compose** (Postgres, Mongo, Mailpit) ; **Maven ; Git ; CI GitHub Actions**.
- Architecture **en couches** :



Développement

Deux fonctionnalités CRUD

- **Feature 1 – Inscription d'un utilisateur** (validation, sécurité, jeu d'essai).
- **Feature 2 – Gestion des cours et leçons** (cycle de vie, autorisations).
- Motif commun : **Contrôleur** → **Service (règles métier)** → **Repository**, erreurs HTTP explicites (**403 / 404 / 409**).

Feature 1 – Inscription (CP4, CP7)

- Formulaire **validé** (Bean Validation), **erreurs accessibles**.
- **Unicité** username/e-mail ; motif **Post-Redirect-Get**.

```
public User register(RegisterForm form) {  
    if (users.existsByUsername(form.username())) {  
        throw new DuplicateUsernameException();  
    }  
    if (users.existsByEmail(form.email())) {  
        throw new DuplicateEmailException();  
    }  
    // hachage BCrypt, jamais de mot de passe en clair  
    user.setPassword(encoder.encode(form.password()));  
    return users.saveAndFlush(user);  
}
```

learn-dev Home Log in [Sign up](#)

Your registration could not be completed.
Check the highlighted field below.

Create an account

Username
3 to 59 characters.

Email

Email already registered

Password
At least 12 characters.

[Create my account](#)

Already registered? [Log in](#)

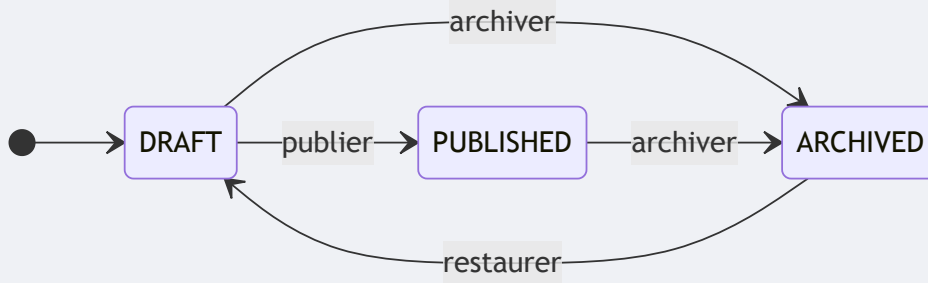
learn-dev — an interactive programming learning platform. [Source on GitHub](#)

Jeu d'essai – Inscription

Scénario	Attendu	Obtenu
Cas nominal	Compte créé, mot de passe haché , rôle STUDENT	Conforme
E-mail déjà utilisé	Erreur de champ, aucune création	Conforme
Mot de passe trop court (< 8)	Rejet par la validation	Conforme
E-mail invalide	Rejet par la validation	Conforme

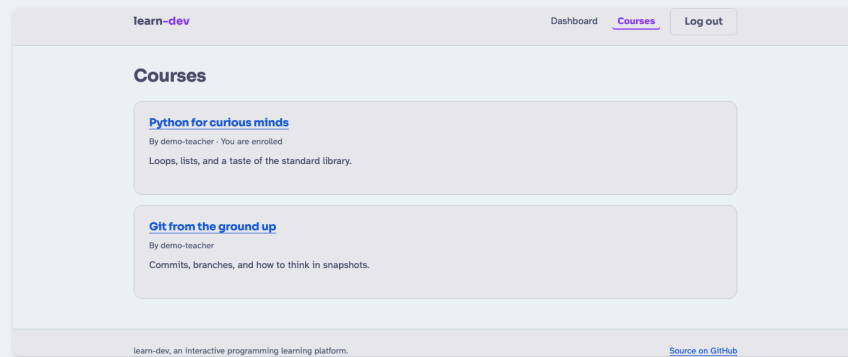
Feature 2 – Cours & leçons (CP7)

- **Autorisations** : le cours d'un autre formateur → **403** ; contenu invisible → **404** ; transition invalide → **409**.
- Réordonnancement des leçons, gestion des inscrits, **journal d'audit**.



Front-end : interfaces & responsive (CP3, CP4)

- 21 gabarits Thymeleaf, HTML5 sémantique, CSS responsive, fragment de mise en page partagé.
- Rendu **dynamique** du Markdown des leçons (converti puis **assaini** contre les XSS).
- Navigation précédent/suivant, affichage conditionnel selon le rôle.



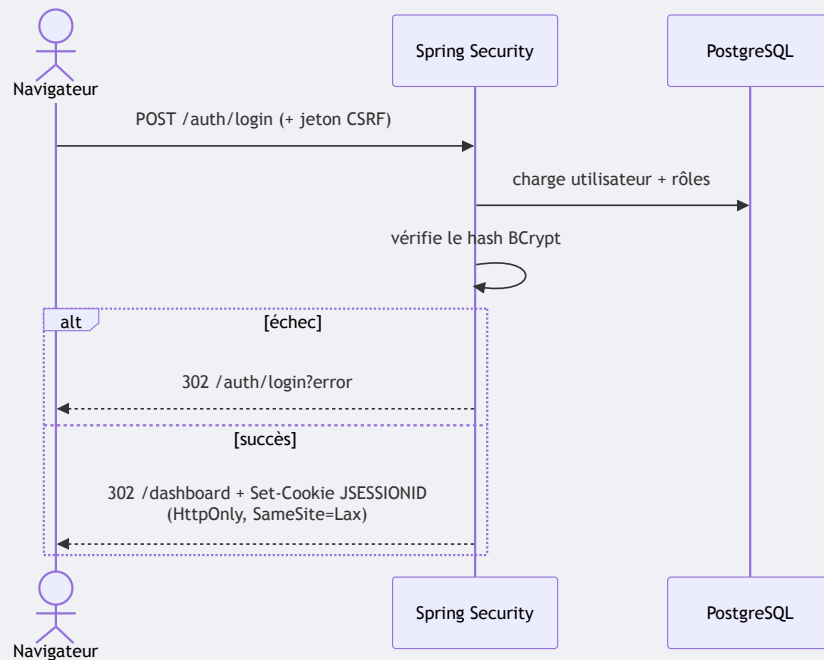
Accès aux données (CP6)

- **Spring Data JPA** : *repositories* = interfaces, requêtes **dérivées** et **paramétrées** (anti-injection SQL) ;
`@Transactional`.
- **NoSQL** : MongoDB **provisionné** en vue d'un stockage de contenu (non encore branché).

```
public interface UserRepository extends JpaRepository<User, UUID> {  
  
    @EntityGraph(attributePaths = "roles")    // l'utilisateur ET ses rôles en 1 requête  
    Optional<User> findByUsername(String username);  
  
    boolean existsByUsername(String username);  
    boolean existsByEmail(String email);  
}
```

Sécurité

- **BCrypt**, **CSRF** activé, **anti-XSS** (jsoup).
- **Anti-IDOR** (UUID), **anti-énumération**, **verrouillage de compte**, **rate-limiting**.
- Jetons **hachés SHA-256**, **journal d'audit**, matrice d'autorisations **testée**.
- Référence **OWASP Top 10** ; veille **Semgrep**.



Tests

- **64 tests** (pyramide) : unitaires (Mockito), tranche JPA, intégration (MockMvc).
- **PostgreSQL réel** en test via **Testcontainers** (schéma fidèle : UUID, contraintes).
- **Couverture** mesurée par **JaCoCo**, publiée sur **Codecov** ; CI à 4 workflows.

Déploiement

Conteneurisation & CI/CD (CP8)

- **Image Docker multi-étapes** : build Maven puis exécution **non-root**.
- **Docker Compose** (Postgres / Mongo / Mailpit) ; profils **dev** / **prod** ; secrets via `.env`.
- **Procédure documentée** :

```
docker compose up -d      # 1. services (base de données, SMTP local)
./mvnw spring-boot:run    # 2. application ⇒ http://localhost:8080
```

- **CI GitHub Actions** : build, tests, lint, contrôle anti-dérive de schéma.

Conclusion

Bilan des compétences

CP	Compétence	Où
CP1	Environnement de travail	Stack, Docker, CI
CP2	Maquetter	Figma, RGAA, responsive
CP3	Interfaces statiques	Thymeleaf, CSS, thèmes
CP4	Interfaces dynamiques	Formulaires, PRG, Markdown
CP5	Base de données	Merise, PostgreSQL, Liquibase
CP6	Accès aux données	Spring Data JPA (+ Mongo)
CP7	Composants métier	Services, règles, erreurs HTTP
CP8	Déploiement	Docker, Compose, CI/CD

Difficultés & évolutions

- **Difficultés** : concurrence à l'inscription (contrainte d'unicité captée) ; sécurité du rendu Markdown (XSS → assainissement) ; un seul `h1` par page (accessibilité).
- **Évolutions** : brancher **MongoDB** (NoSQL) ; suivi de progression leçon par leçon ; API REST.

Merci

Merci de votre attention.

Projet : github.com/ebouchut/learn-dev

Des questions ?